

Boxing and Unboxing in C# with Examples

In this article, I will discuss **Boxing and Unboxing in C#** with Examples. Please read our previous article discussing [Stack, Heap, Value Type, and Reference Type in C#](#) with Examples. Boxing and Unboxing are two fundamental concepts in C# that deal with converting between value and reference types. Understanding these concepts is crucial because they affect performance and memory usage.

So, what we will do in this article is. First, we will try to understand Boxing and Unboxing with examples. Then, we will see how the IL code looks, and finally, we will see the performance implications because of boxing and unboxing in the C# application.

Boxing and Unboxing in C#:

Boxing: Boxing is the process of converting a value type (like int, double, struct) to a reference type (object). When a value type is boxed, a new object is allocated to the heap, and the value is copied into it.

Unboxing: Unboxing is the reverse process of boxing, where a value is extracted from an object. It involves explicitly converting a reference type (object) into a value type. This operation also involves a copy operation, where the value is copied from the heap into the stack.

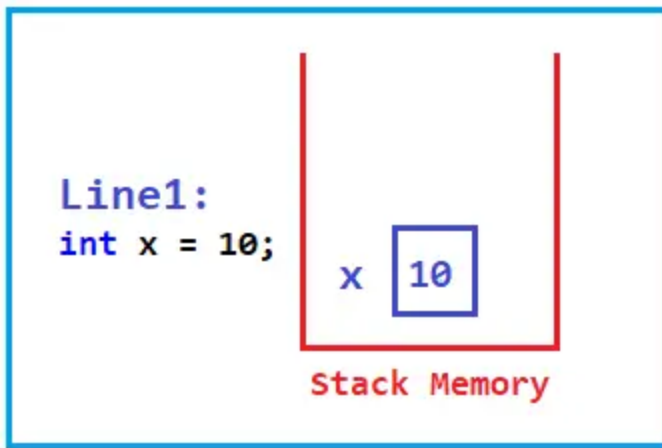
Let us understand Boxing and Unboxing in C# with an example. Please have a look at the following code.

```
public void SomeMethod()
{
    Line1 ➡ int x = 10;
    Line2 ➡ object y = x; //Boxing
    Line3 ➡ int z = (int)y; //Unboxing
}
```

The above method contains three lines of code. Now, let us understand what happens when executing each code line.

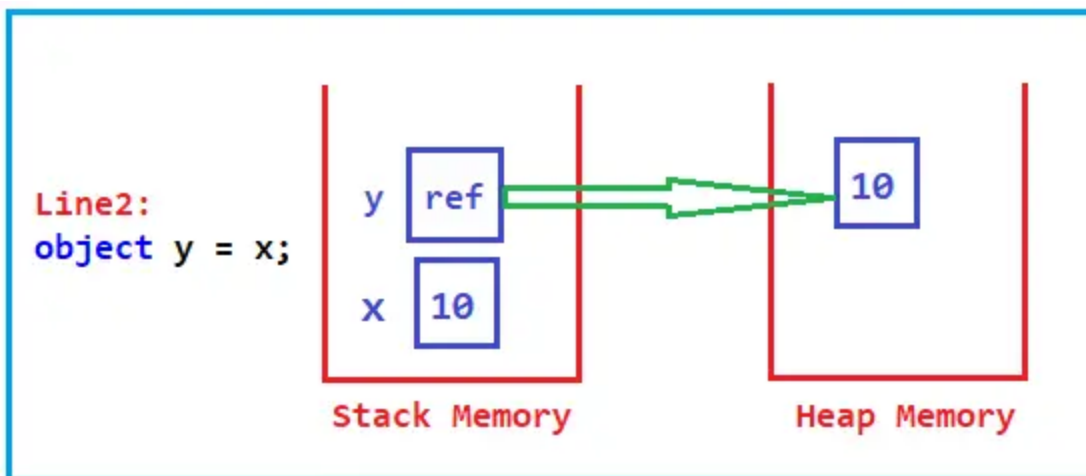
Line1: int x = 10;

When this statement is executed, an integer variable x will be created in the Stack memory with a value of 10. For a better understanding, please have a look at the following diagram.



Line2: object y = x;

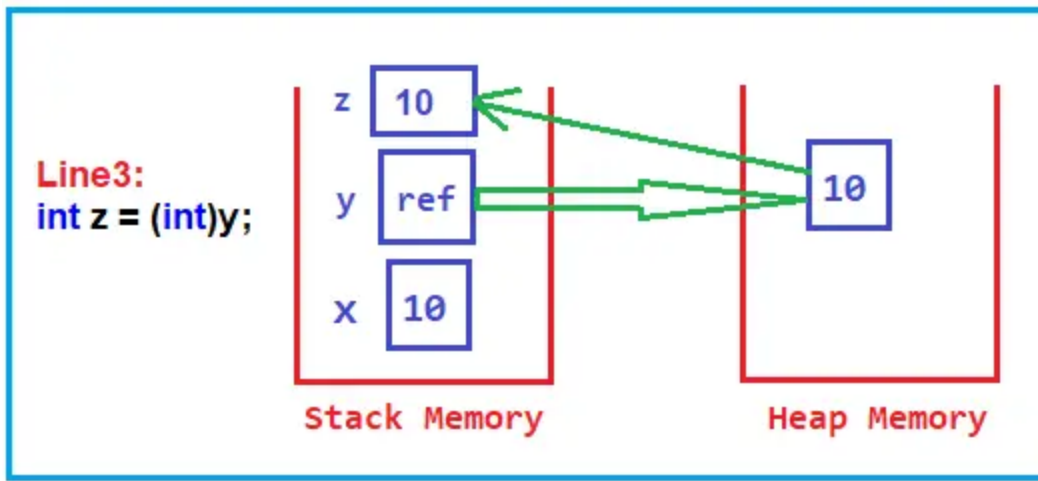
When executing this statement, we move the x value, i.e., 10, to an object data type. If you remember, the object is the parent class for all classes in the .NET Framework. When we move a value type to a reference type, it is called Boxing. So, here we are moving value type integer x to reference type object y, so we are performing boxing here.



So, when we move a value type to a reference type or set a value type to a reference type, it is called Boxing in C#.

Line3: int z = (int)y;

When executing this statement, we move the object value to an integer data type by doing type casting. When we move a reference type to a value type, it is called Unboxing. So, we are moving the reference type value, i.e., y, to an integer type, i.e., z, so we are performing Unboxing here.



So, when we move a reference type to a value type or set it to a value type, it is called Unboxing in C#.

Note: Boxing means you set a value type to a reference type, and unboxing means you set a reference type to a value type.

Example to Understand Boxing and Unboxing in C#:

Now, we will create a simple example implementing the Boxing and Unboxing using C# Language, and then we will see how the IL code looks like. So, create a console application and then modify the Program class as follows:

```
namespace BoxingUnboxingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int x = 10;
            object y = x; //Boxing
            int z = (int)y; //Unboxing
        }
    }
}
```

Now, build the solution and make sure the EXE is generated. In my case, the EXE is generated in the following location.

This PC > DATA (D:) > Projects > BoxingUnboxingDemo > BoxingUnboxingDemo > bin > Debug

Name	Date modified	Type	Size
 BoxingUnboxingDemo	03-07-2022 09:36	Application	5 KB
 BoxingUnboxingDemo.exe.config	03-07-2022 08:52	XML Configuration...	1 KB
 BoxingUnboxingDemo.pdb	03-07-2022 09:36	Program Debug D...	16 KB

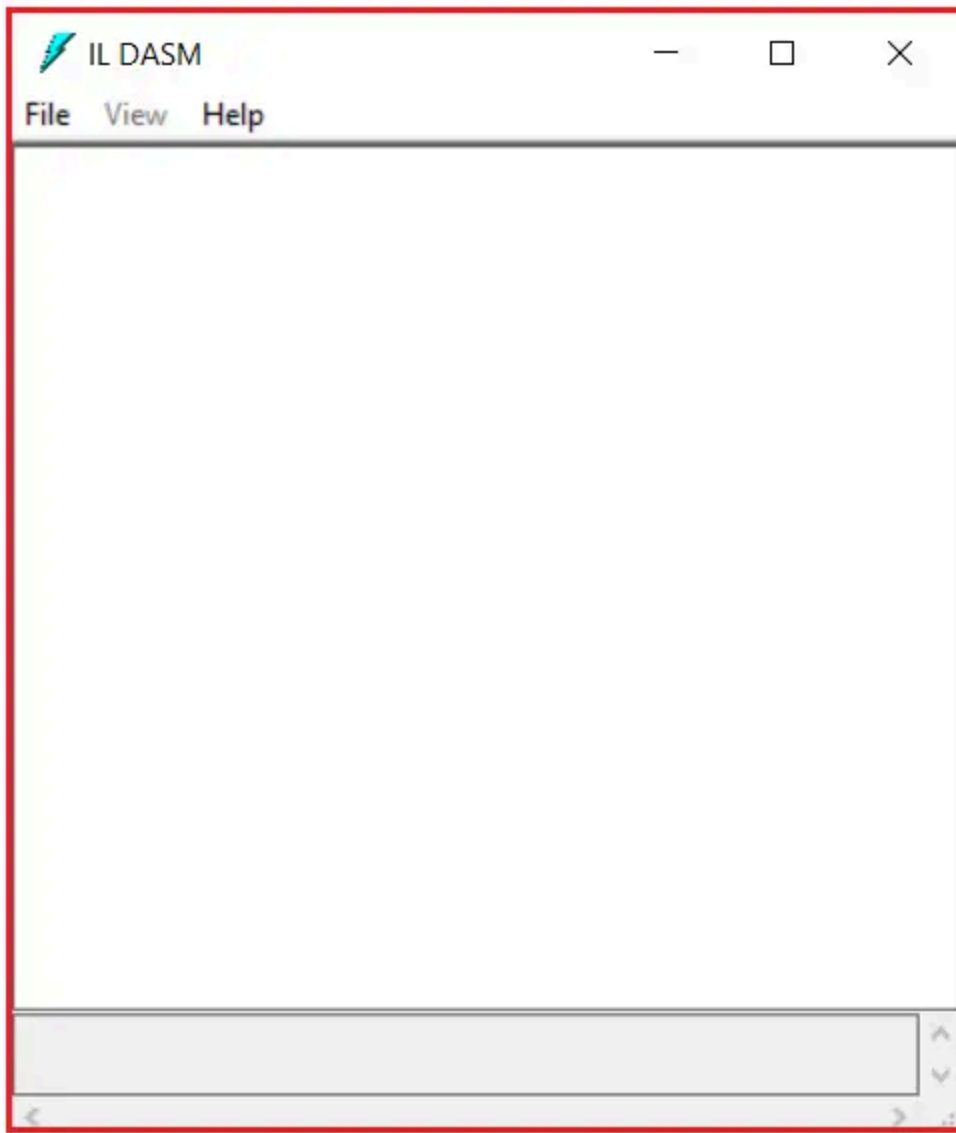
Now, open the Visual Studio Command Prompt, type ILDASM, and press the enter button, as shown in the image below.

```
C:\> Developer Command Prompt for VS 2017

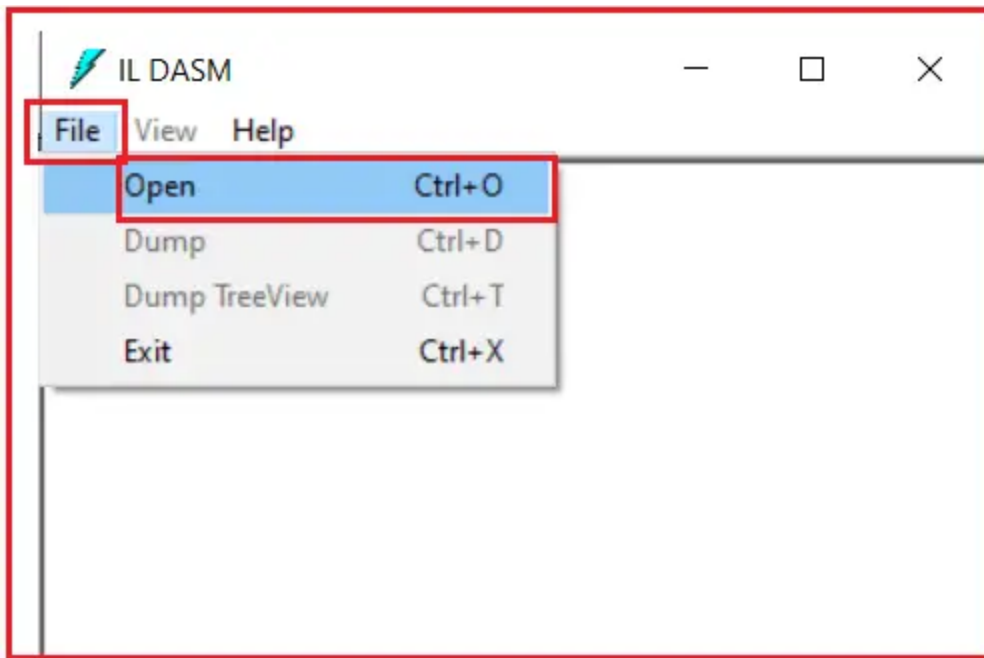
*****
** Visual Studio 2017 Developer Command Prompt v15.9.36
** Copyright (c) 2017 Microsoft Corporation
*****

C:\Program Files (x86)\Microsoft Visual Studio\2017\Community>ildasm
```

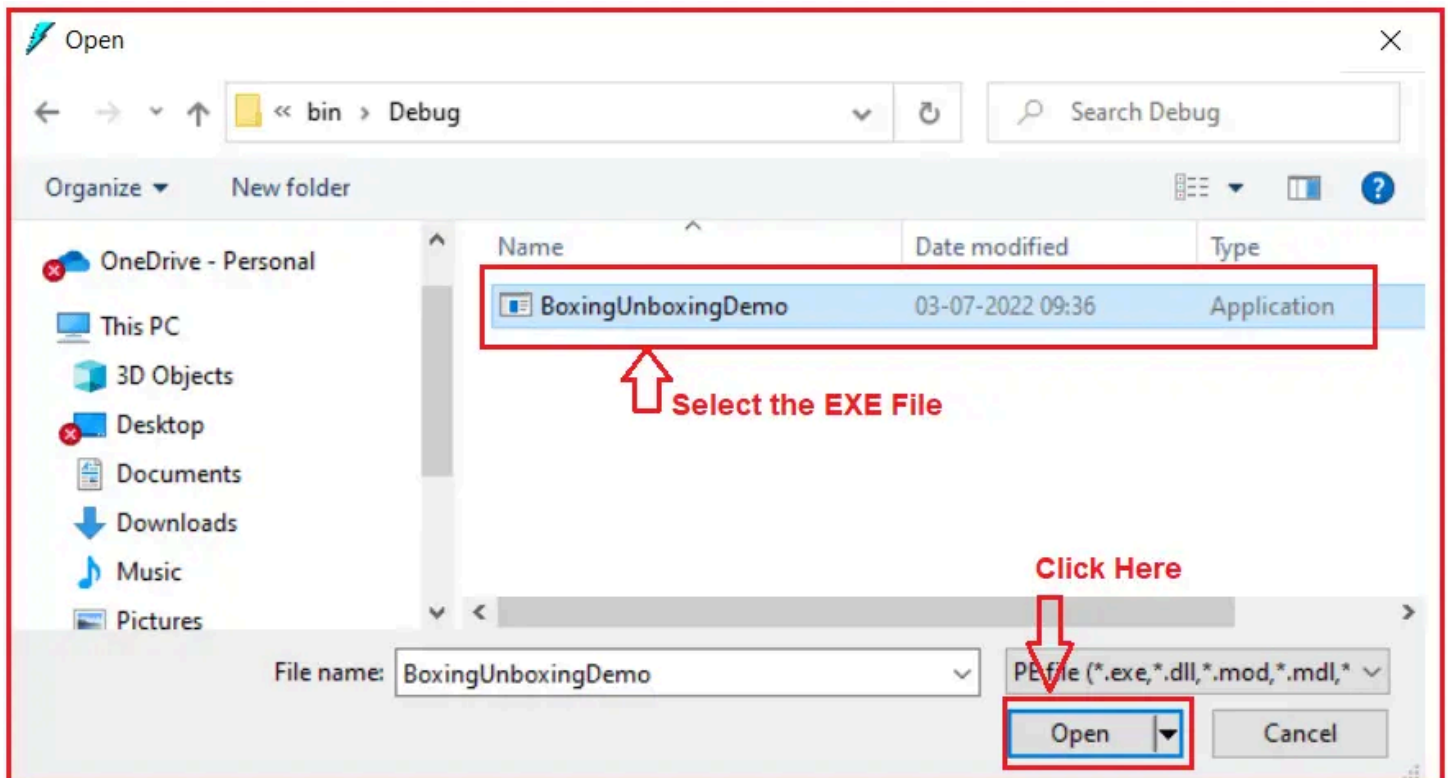
Once you press the enter button, it will open the ILDASM window, as shown in the image below.



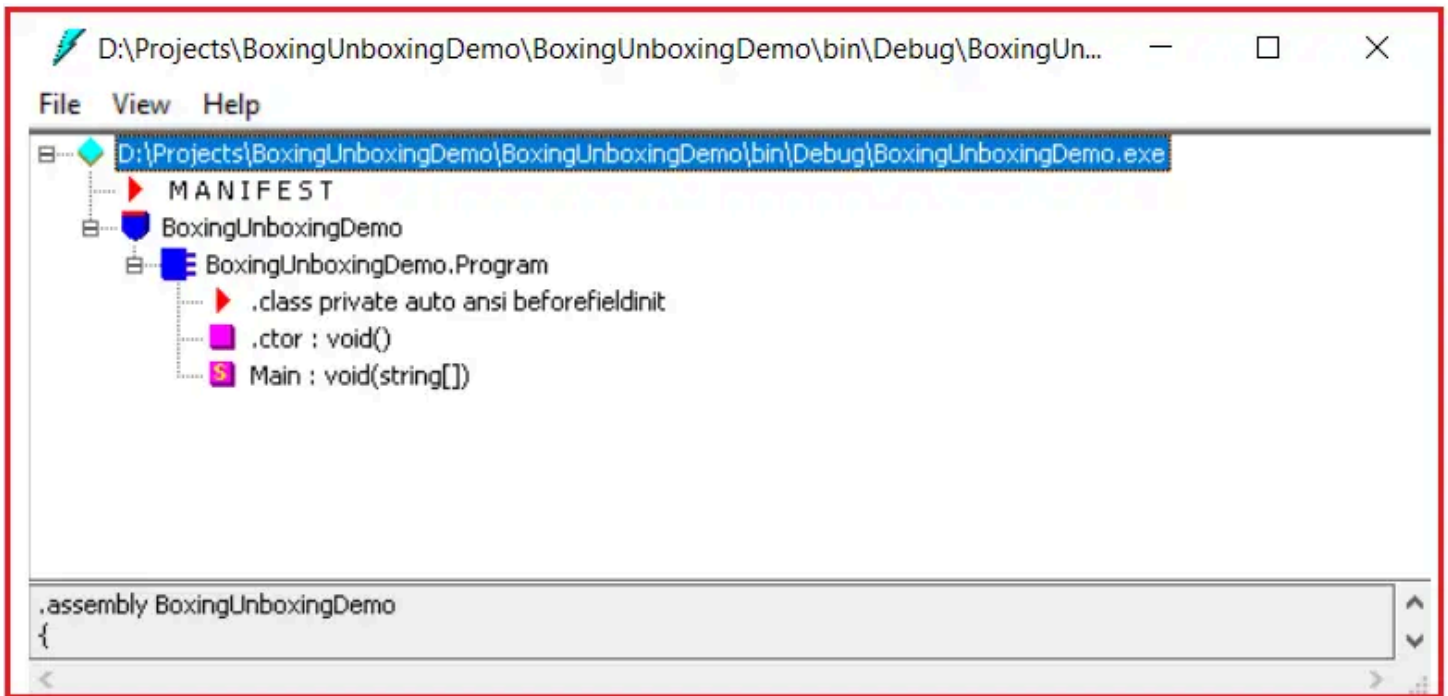
Now, open the EXE file using ILDASM. To do so, select File => Open from the context menu, as shown in the below image.



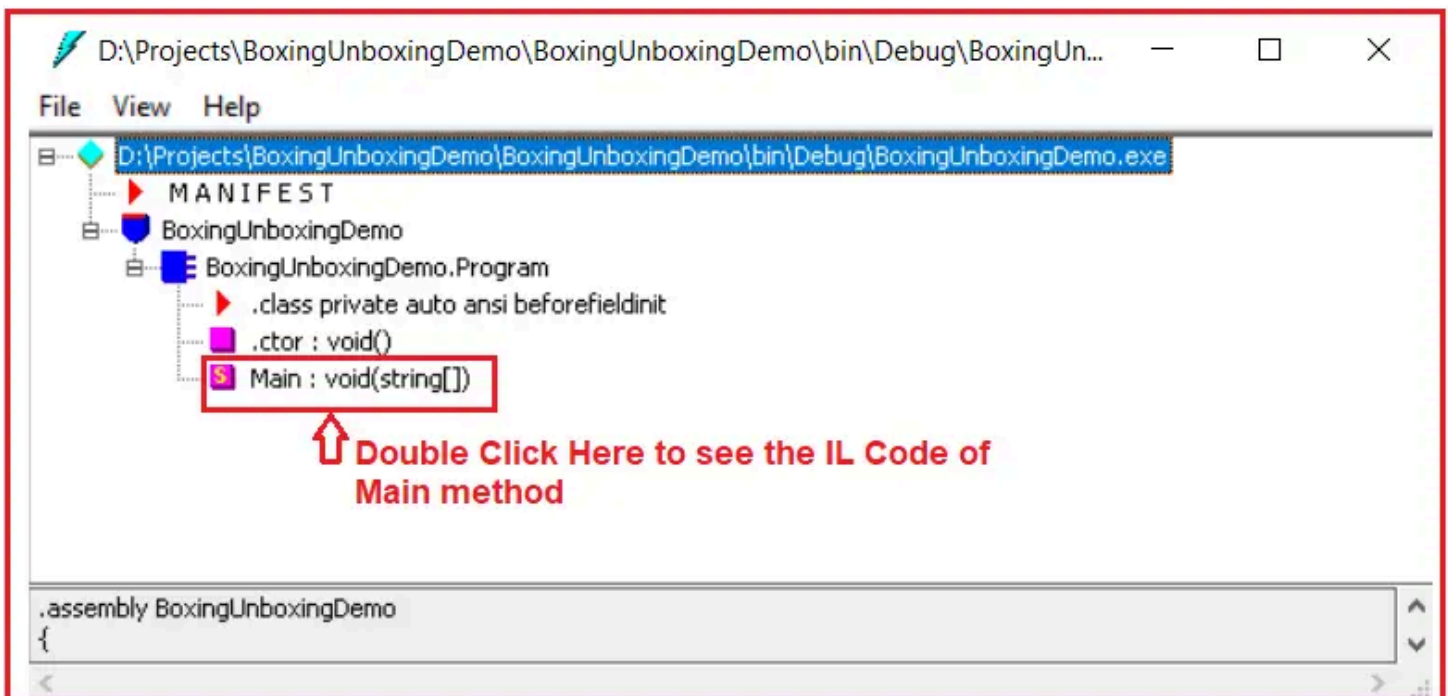
It will open the select EXE window. Select the EXE file from this window and click the Open button, as shown in the image below.



Now, you can see the EXE file is loaded into the ILDASM window. You can expand the section by clicking on the plus button. So, after expanding, you will see the following.



If you remember, we have written our code inside the Main method. So, double-click on the Main method to see the IL code.



Once you double-click, you will see the following IL Code of the Main method.

Now, if you look at the IL code, you will see that there is something called box, which is nothing but boxing, and something called unbox, which is nothing but unboxing. You can also see how the IL code looks like for Boxing and Unboxing in C#. For a better understanding, please have a look at the following image.

So, as of now, we have seen what is boxing and unboxing in C# and how the IL Code looks like for boxing and unboxing. Let us move forward and see if there is any performance implication when we perform boxing and unboxing, i.e. when we move a value type to a reference type and from a reference type to a value type.

Performance Implication of Boxing in C#:

Let us first see the performance implication of Boxing in C#. Please have a look at the following example. In the below example, we have created two methods, i.e., Boxing and WithoutBoxing. In the Boxing method, we are performing boxing, i.e., moving a value type to a reference type, and in the WithoutBoxing method, we are not performing either boxing or unboxing, i.e., we are performing a simple assignment. Then, from the Main method, we are calling both Methods using two different for loops. And both the loop is going to execute **1000000** times. Further, to measure the time, we are using Stopwatch.

```
using System;
using System.Diagnostics;
namespace BoxingUnboxingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Stopwatch stopwatch1 = new Stopwatch();
            stopwatch1.Start();
            for(int i = 1; i<= 1000000; i++)
            {
                Boxing();
            }

            stopwatch1.Stop();
            Console.WriteLine($"Boxing took: {stopwatch1.ElapsedMilliseconds}");
            Stopwatch stopwatch2 = new Stopwatch();
            stopwatch2.Start();
            for (int i = 1; i <= 1000000; i++)
            {
                WithoutBoxing();
            }
            stopwatch2.Stop();
            Console.WriteLine($"Without Boxing took: {stopwatch2.ElapsedMilliseconds}");

            Console.ReadKey();
        }
        //With Boxing
        public static void Boxing()
        {
            int i = 100;
            object j = i; //Boxing
        }
        //Without Boxing
        public static void WithoutBoxing()
        {
            int i = 100;
```

```

        int j = i; //No Boxing and No Unboxing
    }
}

```

Output:

```

Boxing took: 26 MS
Without Boxing took: 18 MS

```

As you can see in the above image, Boxing took 26 MS on my machine, while it took 18 MS without boxing. That means if you use boxing, there is performance degradation in C#.

Performance Implication of Unboxing in C#:

Now, let us see the performance implication of Unboxing in C#. Please have a look at the following example. In the below example, we have created two methods, i.e., Unboxing and WithoutBoxingAndUnboxing. In the Unboxing method, we perform unboxing, i.e., moving a reference type to a value type. In the WithoutBoxingAndUnboxing method, we are not performing either boxing or unboxing, i.e., we are performing a simple assignment. Then, from the Main method, we are calling both Methods using two different for loops. And both the loop is going to execute **1000000** times. Further, to measure the time, we are using Stopwatch.

```

using System;
using System.Diagnostics;
namespace BoxingUnboxingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Stopwatch stopwatch1 = new Stopwatch();
            stopwatch1.Start();
            for(int i = 1; i<= 1000000; i++)
            {
                Unboxing();
            }

            stopwatch1.Stop();
            Console.WriteLine($"Unboxing took: {stopwatch1.ElapsedMilliseconds} MS");
            Stopwatch stopwatch2 = new Stopwatch();
            stopwatch2.Start();
            for (int i = 1; i <= 1000000; i++)
            {
                WithoutBoxingAndUnboxing();
            }
            stopwatch2.Stop();
            Console.WriteLine($"WithoutBoxingAndUnboxing took: {stopwatch2.ElapsedMilliseconds} MS");
            Console.ReadKey();
        }
    }
}

```

```

    }
    //With Unboxing
    public static void Unboxing()
    {
        object j = 100;
        int i = (int) j; //Unboxing
    }
    //Without Boxing
    public static void WithoutBoxingAndUnboxing()
    {
        int i = 100;
        int j = i; //No Boxing and No Unboxing
    }
}

```

Output:

```

Unboxing took: 26 MS
WithoutBoxingAndUnboxing took: 19 MS

```

You can see in the above output that Unboxing took 26 MS on my machine while WithoutBoxingAndUnboxing took 19 MS. That means if you use Unboxing, there is performance degradation in C#.

Note: We should always avoid Boxing and Unboxing in C# due to performance degradation in application development.

Key Points Related to Boxing and Unboxing in C#:

- **Performance Considerations:** Boxing and unboxing are computationally expensive operations. They involve memory allocation and copying, affecting performance, especially in tight loops or high-performance scenarios.
- **Memory Usage:** Boxing allocates memory on the heap, which can lead to increased memory usage and may trigger garbage collection more often.
- **Type Safety:** Unboxing requires an explicit cast and will throw an `InvalidCastException` at runtime if the types do not match.
- **Use in Collections:** Before generics were introduced in .NET 2.0, collections like `ArrayList` could only store objects, so value types were boxed when added to these collections. With generics (`List<T>`, `Dictionary<TKey, TValue>`), boxing can be avoided as the collections can store specific value types.

Best Practices:

- **Avoid Unnecessary Boxing and Unboxing:** Be mindful of your data types, especially in performance-critical code, to avoid unnecessary boxing and unboxing.
- **Use Generics:** Prefer generic collections over non-generic ones to avoid boxing of value types.
- **Understanding Implicit Boxing:** Be aware of situations where boxing can occur implicitly, like passing a value type to a method that accepts an object.

In the next article, I will discuss [Object-Oriented Programming in C#](#) with Examples. In this article, I try to explain **Boxing and Unboxing in C#** with Examples. I hope you enjoy this Boxing and Unboxing in C# with Examples article. I would like to have your feedback. Please post your feedback, questions, or comments about this article.